

From SAP Hybris to Salesforce. Automatically. Verifiably.

SAP Hybris · Java



AI Agent Team



Salesforce · Apex

An AI-powered migration platform that reads a legacy commerce codebase, rewrites it for Salesforce, reviews its own work, and proves it deploys — before a human ever touches it.

Ships as a **VS Code extension** + CLI

Powered by **Anthropic Claude**

63 automated tests

Replatforming by hand is slow, expensive, and quietly loses business logic.

Months → Years

A mid-size Hybris platform holds hundreds of Java classes, a custom data model, scheduled jobs, and live data. Manual rewrites are measured in quarters.

Two-platform experts

It takes engineers senior in *both* Hybris and Salesforce — one of the rarest, most expensive skill combinations in the market.

Silent logic loss

Rules like *“reject orders with a zero total”* vanish in manual translation — and nobody notices until production.

Point it at the old code. Get a working Salesforce project back.

● In — a Hybris codebase

- Java business logic (orders, customers, pricing...)
- The data model (`items.xml`)
- Actual data records (ImpEx files)
- Nightly scheduled jobs



MINUTES-HOURS

● Out — a deployable Salesforce project

- Apex code in Salesforce's own best-practice patterns
- A test suite for every class
- The data model, data, and schedules — migrated
- A confidence report: what to trust, what to review

Four moves — the same way a strong engineering team would do it.

1

Understand

Reads every file, maps which parts depend on which, and writes down the business rules it finds — before writing any code.

3

Build & Review

Writes the Salesforce code and tests; a second AI reviews every piece skeptically before it's accepted.

2

Plan

Decides what each piece should become — including what should *not* be custom code at all.

4

Prove

Deploys to a real Salesforce environment, reads real errors, and fixes them itself — in a loop, until green.

Every decision along the way is written to a plain-language log — **nothing is a black box.**

Not one AI call — four specialists that check each other.

STRATEGY

The Planner

Reads the whole codebase first and sets the strategy for every piece.

"This pricing logic shouldn't be custom code — Salesforce already has a product for that."

EXECUTION

The Builder

Writes the Salesforce code and a matching test for every piece the Planner assigned.

"Built to Salesforce's own enterprise patterns — the way a senior developer would."

QUALITY GATE

The Critic

Reviews every piece adversarially: is the original logic preserved? Is it secure?

"The zero-total rule got lost in translation — sending it back."

PROOF

The Verifier

Deploys to a real Salesforce org and self-heals real errors until it compiles and passes coverage.

"Not 'it looks right' — it actually deployed."

SHARED WHITEBOARD

The team works over a shared record of every decision, finding, and open question — exported as a readable **migration plan** with every run.

It doesn't stop at "the AI wrote code." It proves the code works.

Deploy for real

Dry-run deploy to an actual Salesforce environment

Read real errors

Actual compiler output — not guesses

Fix automatically

Missing field? Add it (with evidence). Broken code? Repair it.

Raise the bar

Below Salesforce's 75% test-coverage requirement? Write more tests.

The loop repeats until the project is **green** — compiled, covered, and scored. Anything the system isn't confident about is **flagged for a human**, never silently shipped.

Grounded

The AI can only reference data fields proven to exist in *your* data model — hallucinations are caught, not shipped.

Reviewed

A second AI independently challenges the first one's work on behavior, security, and standards.

Scored

Every class gets a High / Medium / Low confidence score — reviewers know exactly where to spend their time.

Two moments from an actual migration, unscripted.

THE PLANNER'S JUDGMENT

It refused to write unnecessary code.

Handed a custom discount/promotions engine, the Planner didn't translate it. It recommended **Salesforce CPQ** — the native product built for pricing rules — and generated no custom code for it.

Its own words: **"Discount/promo-code pricing rules are a textbook fit for Salesforce CPQ."** That's architect-level judgment — less code to own, less technical debt.

THE CRITIC'S CATCH

It caught a silently-lost business rule.

The original Java rejected orders with a non-positive total. The first translation *compiled perfectly* — but dropped that check on one path. The Critic caught it and blocked the class as "needs review."

This is exactly the class of bug that slips through manual migrations and surfaces **in production**. Here it was caught before a human ever looked at the code.

What's shipped and working today.

CAPABILITY	WHAT IT COVERS	STATUS
Code migration	Java business logic → best-practice Salesforce Apex, with a test class per class	SHIPPED
Data model	Hybris data definitions → Salesforce objects, fields, dropdowns, relationships, constraints	SHIPPED
Data records	ImpEx files → load-ready CSVs + a safe, re-runnable import runbook	SHIPPED
Scheduled jobs	Hybris cronjobs → Salesforce scheduled Apex, same timing, ready-to-run script	SHIPPED
AI agent team + self-healing verification	Planner · Builder · Critic · Verifier, real-org deploy loop, confidence scoring	SHIPPED
Business processes & storefront APIs	Hybris workflows → Salesforce Flow; storefront REST → Apex REST	ROADMAP

63 automated tests, all passing

3 AI providers – incl. a free, offline test mode

Every run reports its own **cost**

Your code, your keys, your choice of where it goes.



You control the AI provider

Code is sent only to the AI vendor you explicitly configure — nowhere else. Keys live in your settings, never in source control or the shipped product.



A zero-exposure test mode

"Mock" mode runs the entire pipeline with **nothing leaving your machine** — for sensitive codebases and free evaluation.



Secure code by default

Generated Apex enforces Salesforce field-level security and record-sharing rules as a house standard — not as an afterthought.



Nothing destructive, ever

Deployments are validation-only dry runs. Every AI decision is logged and auditable. A human holds the final deploy button.

On the roadmap: private/VPC model hosting and approval-gated review workflows for regulated environments.

What changes, concretely.

WITHOUT THE PLATFORM	WITH THE PLATFORM
Months of manual rewriting	A working first draft in minutes–hours
Business rules silently lost	A dedicated AI reviewer checks rule preservation; a parity score tracks it
“Trust me, it compiles”	Deployed to a real org and self-corrected until verifiably green
Everything becomes custom code to maintain forever	The AI recommends native Salesforce products where they fit — less debt
A black-box engagement	A decision log and confidence score on every single class

The honest positioning: this produces a **strong, verified first draft plus a prioritized review list** — it makes expert reviewers dramatically faster; it doesn’t remove them.

The questions everyone asks.

Q Does this replace our developers?

No — it replaces the *grind*. Experts review a scored first draft instead of rewriting from scratch, and the confidence scores tell them exactly where to look.

Q Is our source code safe?

It goes only to the AI provider you configure — or nowhere at all in mock mode. No secrets are ever bundled or committed; deploys are dry-run only.

Q What can't it do yet?

Hybris business-process workflows, storefront REST APIs, and automated CPQ setup are on the near-term roadmap. It tells you honestly what it didn't migrate — and why.

Q What if the AI makes something up?

Three nets: it can only use data fields proven to exist; a second AI reviews every class; and the code must actually compile against a real Salesforce org.

Q What does a run cost?

Every report itemizes its own AI usage. Caching keeps repeat costs low, and cheaper models handle the simple steps automatically.

Q Can we try it on our codebase?

Yes — today. Start in free mock mode to validate parsing with zero exposure, then run a real slice with an API key. It's a right-click in VS Code.

Built in phases — each one already de-risking the next.

PHASE 0 ✓

Prove correctness

Self-healing deploys, confidence scoring, behavior-parity checks

PHASE 1 ✓

The agent team

Planner, Builder, Critic, Verifier over a shared whiteboard

PHASE 2 ◉

The whole platform

Data ✓ · schema ✓ · scheduled jobs ✓ · processes & storefront APIs next

PHASE 3

Enterprise-grade

Review workspace, audit trail, private/VPC models, org-aware reuse

PHASE 4

It learns

Every migration makes the next better and cheaper; assessment product

One axis runs through every phase: moving from output you have to *trust* → to output you can **verify**.

THE ASK

Give us one real slice of a Hybris codebase. We'll hand back verified Salesforce — and the receipts.

A pilot costs days, not months. Free mock mode first, a scored real run second, your team's verdict third.

Live demo available **today**

Full docs: **PRD** · **TDD** · **flows** · **security**

Pilot-ready